

Padrao Builder

"Separar a construcao de um objeto complexo da sua representacao, de modo que o mesmo processo de construcao possa criar diferentes representacoes."

— Gang of Four, *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994

DISCIPLINA

Padroes de Projeto de
Software

INSTITUICAO

Universidade de Fortaleza —
UNIFOR

EQUIPE

Gustavo Q.
Mateus

APLICADO EM

AgroPlace — Backend Java 17 / Spring
Boot 3

SOBRE O PADRAO

O que e o Padrao Builder?

Classificacao GoF: Padrao Criacional (Creational Pattern)
Definido originalmente por Gamma, Helm, Johnson e Vlissides no livro de 1994.

O Builder separa o **algoritmo de construcao** de um objeto complexo da sua **representacao final**. Isso permite que o mesmo processo de montagem produza diferentes variantes de um objeto.

Diferente de uma *factory* que entrega o objeto em uma unica chamada, o Builder monta o objeto **passo a passo** — cada etapa configura uma parte especifica — e so materializa o produto ao chamar `construir()`.

Factory Method

Delega a criacao a subclasses

Builder

Monta passo a passo
nosso foco

Abstract Factory

Familias de objetos

OS 4 PARTICIPANTES GOF

Builder

Interface que declara os metodos de configuracao para cada parte do produto. Define o contrato de construcao.

ConcreteBuilder

Implementa a interface Builder, acumula o estado interno e executa validacoes em `construir()`.

Director

Orquestra a sequencia de chamadas ao Builder. Conhece o processo, nao o produto resultante.

Product

O objeto complexo sendo construido. Seu construtor e restrito — so pode ser criado via Builder.

QUANDO APLICAR

Casos de uso do Padrao Builder

01

Objetos com muitos parametros

Quando um objeto exige 5 ou mais parametros no construtor, o Builder substitui a lista confusa por uma API fluente e nomeada.

02

Parametros opcionais

Evita passagem explicita de null para campos opcionais. Apenas os metodos necessarios sao chamados.

03

Validacao centralizada

O metodo `construir()` e o unico ponto de validacao — garante que nenhum objeto invalido seja criado.

04

Multiplas representacoes

O mesmo Diretor pode usar diferentes ConcreteBuilders para produzir variantes do mesmo tipo de produto.

EXEMPLOS NO ECOSSISTEMA JAVA

`StringBuilder` · `HttpClient.newBuilder()` · `Stream.builder()` ·
`Locale.Builder`

EXEMPLOS NO SPRING

`UriComponentsBuilder` · `MockMvcRequestBuilders` ·
`SecurityFilterChain` via lambda

O PROBLEMA

Anti-padrão: telescoping constructor

No AgroPlace, a entidade Anuncio exige **12 parametros**. Sem Builder, o código ficaria assim:

```
// ANTI-PADRAO – construtor telescopico
new Anuncio(
    conta,
    "Lote Nelore Macho", // titulo
    "BOVINO",           // especie
    "Nelore",           // raca
    50,                 // quantidade
    new BigDecimal("420.00"), // peso
    24,                 // idadeMeses
    SexoAnimal.MACHO,
    new BigDecimal("3800.00"), // preco
    "Sobral",
    "CE",
    "Animais vacinados, GTA inclusa"
);
```

Problemas concretos desta abordagem no sistema:

Illegibilidade	Parametro 5 e quantidade ou peso? Sem IDE, impossível saber a ordem.
Sem validacao	Nenhum ponto central para checar se preco > 0 ou se estado tem exatamente 2 caracteres antes de persistir.
Acoplamento	Qualquer adicao de campo exige atualizar toda chamada. No AgroPlace ha 3 módulos com objetos de 7 a 14 campos.
Campos opcionais	Campos como raca e descricao forcam null explicito na assinatura do construtor.
Objeto invalido	Um Anuncio com preco negativo ou estado vazio pode chegar ao repositorio.save() sem nenhuma barreira.

Builder em 3 modulos do backend

MODULOS COM BUILDER

autenticacao

ContaUsuario — 7 campos (nome, email, telefone, documento, nomeEmpresa, senhaHash, tipoConta)

anuncio

Anuncio — 12 campos (especie, raca, peso, idade, sexo, quantidade, preco, cidade, estado...)

perfil

PerfilProdutor, PerfilTransportadora e PerfilCorporativa — 3 builders distintos para 3 tipos de usuario

15

arquivos Builder

5

interfaces Builder

5

ConcreteBuilders

5

Directors

FLUXO COMPLETO — CRIACAO DE ANUNCIO

ControladorAnuncio → **ServicoAnuncioPadrao** → **DiretorAnuncio** → **AnuncioConcretoBuilder** → **Anuncio**

```
// ServicoAnuncioPadrao.java
@Override
public RespostaAnuncio criar(
    ContaUsuario conta,
    RequisicaoAnuncio req) {

    DiretorAnuncio diretor =
        new DiretorAnuncio(
            new AnuncioConcretoBuilder()
        );

    Anuncio anuncio =
        diretor.construirDaRequisicao(conta, req);

    return montarResposta(
        repositorio.save(anuncio)
    );
}
```

O servico de aplicacao nao conhece nenhum campo do Anuncio — delega ao Diretor, que conhece o processo de montagem.

Interface, ConcreteBuilder e Director

ANUNCIOBUILDER.JAVA — INTERFACE

```
public interface AnuncioBuilder {  
    AnuncioBuilder conta(ContaUsuario conta);  
    AnuncioBuilder titulo(String titulo);  
    AnuncioBuilder especie(String especie);  
    AnuncioBuilder raca(String raca);  
    AnuncioBuilder quantidade(int qtd);  
    AnuncioBuilder pesoMedioKg(BigDecimal v);  
    AnuncioBuilder idadeMediaMeses(Integer v);  
    AnuncioBuilder sexo(SexoAnimal sexo);  
    AnuncioBuilder precoUnitario(BigDecimal v);  
    AnuncioBuilder cidade(String cidade);  
    AnuncioBuilder estado(String estado);  
    AnuncioBuilder descricao(String desc);  
    Anuncio construir();  
}
```

ANUNCIOCONCRETOBUILDER.JAVA — VALIDAR()

```
private void validar() {  
    if (conta == null)  
        throw new IllegalStateException(  
            "Conta e obrigatoria");  
    if (titulo == null || titulo.isBlank())  
        throw new IllegalStateException(  
            "Titulo e obrigatorio");  
    if (quantidade < 1)  
        throw new IllegalStateException(  
            "Quantidade minima: 1");  
    if (precoUnitario.compareTo(  
        BigDecimal.ZERO) <= 0)  
        throw new IllegalStateException(  
            "Preco deve ser positivo");  
    if (estado.length() != 2)  
        throw new IllegalStateException(  
            "Estado invalido (use UF)");  
}
```

O construtor de Anuncio e **package-private**: so o ConcreteBuilder pode instanciar o produto — nenhum codigo externo cria um Anuncio invalido.

ContaUsuarioBuilder e uso no Servico

CONTAUSUARIOBUILDER.JAVA — INTERFACE

```
public interface ContaUsuarioBuilder {
    ContaUsuarioBuilder nome(String nome);
    ContaUsuarioBuilder email(String email);
    ContaUsuarioBuilder telefone(String tel);
    ContaUsuarioBuilder documento(String doc);
    ContaUsuarioBuilder nomeEmpresa(String ne);
    ContaUsuarioBuilder senhaHash(String sh);
    ContaUsuarioBuilder tipoConta(TipoConta t);
    ContaUsuario construir();
}
```

TIPOCONTA.JAVA — ENUM

```
public enum TipoConta {
    PRODUTOR,
    TRANSPORTADORA,
    CORPORATIVA
}
```

SERVICOAUTENTICACAOPADRAO.JAVA — CADASTRAR()

```
DiretorContaUsuario diretor =
    new DiretorContaUsuario(
        new ContaUsuarioConcretoBuilder()
    );

String nomeEmpresa = requisicao
    .tipoConta() == TipoConta.PRODUTOR
    ? null
    : requisicao.nomeEmpresa();

ContaUsuario conta =
    diretor.construirConta(
        requisicao.nome(),
        requisicao.email(),
        requisicao.telefone(),
        requisicao.documento(),
        nomeEmpresa,
        codificadorSenha.encode(
            requisicao.senha(),
            requisicao.tipoConta()
        )
    );

return montarResposta(
    repositorio.save(conta)
);
```

Tres builders para tres perfis de usuario

TRES PERFIS, TRES BUILDERS

PerfilProdutorBuilder

nomePropriedade, CAR, documentoRural,
inscricaoEstadual, areaHectares, cep, endereco, cidade,
estado, descricao

PerfilTransportadoraBuilder

razaoSocial, CNPJ, RNTRC, tiposVeiculo,
capacidadeCargaKg, possuiTransporteVivo,
responsavelOperacional

PerfilCorporativaBuilder

razaoSocial, CNPJ, segmento, responsavelComercial,
telefoneComercial, site, nomeFantasia

Cada tipo de usuario tem campos especificos do seu dominio. O Builder garante que cada perfil seja montado com as regras corretas sem compartilhar logica.

PERFILPRODUTORCONCRETOBUILDER.JAVA — CONSTRUIR() + VALIDAR()

```
@Override
public PerfilProdutor construir() {
    validar();
    return new PerfilProdutor(this);
}

private void validar() {
    if (conta == null)
        throw new IllegalStateException(
            "Conta e obrigatorio");
    if (nomePropriedade == null
        || nomePropriedade.isBlank())
        throw new IllegalStateException(
            "Nome da propriedade e obrigatorio");
    if (cidade == null
        || cidade.isBlank())
        throw new IllegalStateException(
            "Cidade e obrigatorio");
    if (estado == null
        || estado.length() != 2)
        throw new IllegalStateException(
            "Estado invalido");
}
```


VANTAGENS

Por que Builder é ideal para o AgroPlace



Legibilidade do código

Cada campo é nomeado explicitamente na chain fluente. Não há dúvida se o 4o argumento é peso ou quantidade.



Validação fail-fast

`construir()` é a única porta de saída. Nenhum objeto inválido chega ao repositório — regras de negócio protegidas na camada de domínio.



Campos opcionais sem null

Campos como `raca` e `descricao` simplesmente não são chamados. Sem sobrecarga de construtores e sem `null` explícito.



Encapsulamento total do produto

O construtor de `Anuncio` é `package-private`. Impossível criar um produto malformado fora do Builder.



Extensibilidade

Adicionar um campo ao `Anuncio` requer apenas um novo método na interface e no `ConcreteBuilder`. O `Director` e o `Servico` não mudam — Open/Closed Principle.



Testabilidade

Cada `ConcreteBuilder` pode ser testado isoladamente. É possível criar um `MockBuilder` implementando a interface para testes de `Diretor` sem dependências reais.



Separação de responsabilidades

O `Servico` cuida das regras de negócio. O `Diretor` cuida do processo de montagem. O `ConcreteBuilder` cuida dos campos e validações. Clean Architecture respeitada.



Consistência entre módulos

Os 3 módulos (`autenticacao`, `anuncio`, `perfil`) seguem exatamente o mesmo padrão. Qualquer desenvolvedor que entra no projeto entende imediatamente a estrutura.

PRINCIPIOS SOLID

Builder e SOLID — como se complementam

PRINCIPIO	COMO O BUILDER APLICA NO AGROPLACE	EVIDENCIA NO CODIGO
S — SRP	Builder cuida exclusivamente da montagem e validacao. Servico cuida da regra de negocio. Director cuida do processo. Cada classe tem uma unica razao para mudar.	<code>ServicoAnuncioPadrao</code> nao conhece campos de <code>Anuncio</code>
O — OCP	Para adicionar <code>fotoPrincipal</code> ao <code>Anuncio</code> , basta um novo metodo na interface e no <code>ConcreteBuilder</code> — sem alterar <code>Director</code> ou <code>Servico</code> .	Interface <code>AnuncioBuilder</code> define contrato extensivel
L — LSP	Qualquer novo <code>ConcreteBuilder</code> que implemente <code>AnuncioBuilder</code> pode substituir <code>AnuncioConcretoBuilder</code> no <code>Director</code> sem quebrar comportamento.	<code>DirectorAnuncio</code> depende apenas da interface
I — ISP	Cada Builder tem apenas os metodos do seu produto. <code>ContaUsuarioBuilder</code> nao tem metodos de <code>AnuncioBuilder</code> — interfaces enxutas e coesas.	5 interfaces distintas para 5 entidades
D — DIP	<code>DirectorAnuncio</code> recebe um <code>AnuncioBuilder</code> (interface) no construtor, nao um <code>ConcreteBuilder</code> . Depende da abstracao, nao da implementacao.	<code>new DiretorAnuncio(new AnuncioConcretoBuilder())</code>

COMPARATIVO

Antes e depois — impacto real no código

ANTES — TELESCOPING CONSTRUCTOR

```
// 12 parametros posicionais
// sem validacao, sem nomes
new Anuncio(
    conta, titulo, especie,
    raca, quantidade, peso,
    idade, sexo, preco,
    cidade, estado, descricao
);
```

- Objeto invalido pode ser criado
- Ordem dos parametros causa bugs silenciosos
- Campos opcionais forcaram null explicito
- Leitura impossivel sem IDE
- Qualquer novo campo quebra todas as chamadas

DEPOIS — BUILDER + DIRECTOR

```
// Fluent API nomeada
// validar() chamado em construir()
diretor.construirDaRequisicao(
    conta, req
);

// internamente:
builder.conta(conta)
    .titulo(req.titulo())
    .especie(req.especie())
    .precoUnitario(req.precoUnitario())
    .construir(); // valida aqui
```

- + Validacao centralizada em validar()
- + Campos nomeados — legibilidade total
- + Campos opcionais simplesmente omitidos
- + Produto invalido nunca sai do Builder
- + Novos campos nao quebram o Servico

CONCLUSAO

Builder no AgroPlace — resumo

Problema resolvido

Os 3 modulos criavam objetos com 7 a 12 parametros posicionais. O Builder substituiu por uma API fluente nomeada, segura e extensivel.

Validacao garantida

`validar()` no `ConcreteBuilder` garante que nenhum objeto invalido chega ao `repositorio.save()` — fail-fast antes da persistencia.

Arquitetura limpa

Padrao aplicado em sincronia com Clean Architecture — Builder vive no dominio, Diretor na aplicacao, sem vazamento de dependencias.

NUMEROS DO PROJETO

3

modulos com Builder

15

arquivos dedicados ao padrao

5

principios SOLID atendidos

0

construtores telescopicos no dominio

O padrao Builder e a escolha natural quando um objeto requer muitos parametros de inicializacao, quando alguns sao opcionais e quando a validade do estado e pre-condicao critica para a persistencia.